Design-Notiz: Periodizität der gerouteten Schaltung und die Stabilität des Schnitts

Status: vor der ersten Rechnung geschrieben. Nichts in dieser Notiz darf nach dem ersten Lauf geändert werden, außer in einem mit Datum versehenen Nachtrag.

Vorgeschichte. Die Messung „der gatteroptimale Schnitt wandert nicht, in einundzwanzig von einundzwanzig Graphen" (`gate_vs_state_partition.py`) wurde auf dem **geschriebenen** Interaktionsgraphen gerechnet. Daniel Hug hat gefragt, ob sie das Routing überlebt. Die Antwort war: nicht nur die Identität des Schnitts steht auf dem Spiel, sondern die Begründung — in QAOA legt jede Schicht dieselben Gatter an, und genau diese Schichtperiodizität zerstört das Routing, weil SWAPs die Zuordnung in die nächste Schicht weitertragen. Daniels Gegenvorschlag: Periode eins per Rück-SWAPs erzwingen. Die hier zu prüfende Behauptung: Periodizität stellt sich ohne Rück-SWAPs von selbst ein, nur mit größerer Periode.

1 DEFINITIONEN

Logische Schicht. Eine QAOA-Schicht: der vollständige Satz ZZ-Gatter des Problemgraphen plus der Mischer. Für alle Schichten identisch.

Layout m. Bijektion logisches Qubit → physikalisches Qubit.

Routing-Abbildung f. Gegeben ein Layout m und die feste Gatterliste einer Schicht, erzeugt ein deterministischer Router mit festem Seed einen gerouteten Block $B(m)$ und ein Ausgangslayout $f(m)$. **Entscheidende Voraussetzung:** Jede Schicht wird als Block aus dem jeweils aktuellen Layout geroutet, ohne Lookahead über die Schichtgrenze.

Orbit. $m(0)$, $m(1) = f(m(0))$, $m(2) = f(m(1))$, … Der Layoutraum ist endlich, f ist eine Abbildung einer endlichen Menge in sich; also existieren ein Vorlauf $t \geq 0$ und eine Periode $d \geq 1$ mit $m(t+d) = m(t)$. Das ist kein empirischer Befund, sondern Arithmetik. Gemessen werden die **Werte** von t und d.

Gerouteter Interaktionsgraph $G(k)$. Multigraph auf den logischen Qubits, eine Kante je Zwei-Qubit-Gatter in $B(m(k))$ — ZZ-Gatter **und** SWAPs —, auf logische Bezeichner zurückgerechnet.

Gatterschnitt. Balancierte Bipartition der logischen Qubits, die die Zahl der kreuzenden Zwei-Qubit-Gatter in $G(k)$ minimiert. Bei $n \leq 12$ wird vollständig aufgezählt (für $n = 12$: 462 Partitionen), also exakt und ohne Heuristik.

2 WAS BERICHTET WIRD

Pro Tripel (Problemgraph, Topologie, Seed):

1. t und d
2. ob die Argmin-Partition über $k = t \ldots t+d-1$ konstant ist — und, getrennt davon, über den Vorlauf $k = 0 \ldots t-1$
3. Schnittkosten je Schicht und die Marge (bester gegen zweitbesten Schnitt) je Schicht
4. SWAP-Zahl je Block

5. die Zahl der **zusätzlich** kreuzenden Gatter, die allein von SWAPs kommen

Pooled und innerhalb der Graphfamilien, immer im selben Ausdruck (Simpson-Prüfung).

3 HYPOTHESEN, SAMT NULLBEFUND, VORAB BENANNT

H1 — die Periode ist brauchbar klein. Erwartung: $d \leq 4$ für die geprüften Topologien. *Nullbefund:* d ist groß oder innerhalb p_max wird kein Zyklus gefunden. Dann ist die Periodizität wahr und nutzlos, und genau das ist das Ergebnis.

H2 — der gatteroptimale Schnitt ist über den Zyklus konstant. *Nullbefund:* Er wandert. Dann überträgt sich die 21-von-21 nicht auf die geroutete Schaltung, und das steht im Artikel im Fließtext, nicht in einer Fußnote. Dieser Nullbefund ist der interessantere Ausgang, nicht der schlechtere.

H3 — der freie Orbit ist billiger als erzwungene Periode eins. Erwartung: Die zusätzlichen kreuzenden Gatter über d Schichten liegen unter den Kosten der Rück-SWAPs, die Daniels Verfahren pro Schicht zahlt. *Nullbefund:* Erzwingen ist billiger. Dann ist Daniels Verfahren das richtige, und der freie Orbit bleibt nur eine billigere Diagnose. Die Zahl für die Rück-SWAPs kommt von ihm; bis sie da ist, wird H3 nicht beantwortet, sondern offen gelassen.

4 KONTROLLEN

All-to-all als Pipeline-Prüfung. Auf vollständiger Kopplung werden keine SWAPs eingefügt. Dann *muss* $t = 0$, $d = 1$ herauskommen, und der Schnitt muss dem ungerouteten Gatterschnitt gleichen. Tut er es nicht, ist die Pipeline kaputt und kein Ergebnis gültig. Diese Kontrolle läuft zuerst.

Seed-Kontrolle. Mindestens fünf Router-Seeds je Tripel. Hängen t, d oder die Konstanz des Schnitts am Seed, dann ist das Ergebnis eine Aussage über den Router und nicht über die Schaltung. Per-Seed berichten, nicht mitteln.

Permutations-Kontrolle. Dieselbe Rechnung mit einer Folge **zufälliger** Permutationen gleicher Länge statt des Routers. Ist der Schnitt auch dort konstant, sagt Konstanz über Routing nichts aus.

Vorlauf getrennt vom Zyklus. Die Schnitte des Vorlaufs dürfen nicht in die Zyklus-Statistik einfließen. Der Vorlauf entspricht den „toten Schichten" der Trotter-Messung und braucht dieselbe Behandlung: ausgewiesen, nicht vermischt.

5 STATISTISCHER BODEN

Für ein festes Tripel ist dies eine **deterministische Rechnung**, keine Stichprobe: auf t und d gehören keine Konfidenzintervalle. Die Streuung lebt über Graphen, Topologien und Seeds. Berichtet wird daher in Anzahlen („k von N Graphen"), und jede Aussage über Graphfamilien hinweg trägt die familieninternen Anzahlen mit.

Mindestgitter: **21 Problemgraphen** (dieselben wie bisher, damit die Zahlen vergleichbar bleiben) × **4 Topologien** × **5 Seeds**.

Topologien: heavy-hex, Quadratgitter, Linie, all-to-all (Kontrolle). Die Linie ist der ungünstigste Fall — die Umkehrpermutation kostet dort am meisten — und gehört genau deshalb dazu.

6 ABBRUCHREGEL

p_max = **64 Schichten** je Lauf. Erscheint bis dahin kein Zyklus, wird „kein Zyklus innerhalb 64" vermerkt und abgebrochen. p_max wird **nicht** erhöht, um einen Zyklus zu finden.

Die Messung ist beendet, wenn das Gitter aus Abschnitt 5 vollständig ist — nicht, wenn ein Ergebnis interessant aussieht. Kein Nachladen weiterer Graphen, Topologien oder Seeds nach dem ersten Blick auf die Zahlen.

7 WAS DIE MESSUNG UNGÜLTIG MACHEN WÜRDE

Ein Router mit Lookahead. Routet der Router die ganze Schaltung in einem Zug, sieht er über Schichtgrenzen hinaus; dann ist f keine Funktion des Layouts allein und das Periodizitätsargument greift nicht. Die Messung beschreibt dann ausschließlich das Regime „schichtweises Blockrouting", und diese Grenze steht im ersten Absatz des Berichts, nicht am Ende.

Stochastisches Routing. Qiskits SabreSwap ist zufallsbehaftet. Ein fester Seed macht es für diesen Zweck deterministisch, aber das Ergebnis ist dann an den Seed gebunden — daher die Seed-Kontrolle, und daher wird nie ein einzelner Seed zitiert.

8 WAS DIE MESSUNG NICHT MISST

Nichts über den Quantenzustand. Nichts über Kalibrierung, Drift oder Fidelity. Nichts über Maschinen brauchbarer Größe. Es ist eine rein strukturelle Rechnung auf gerouteten Gatterlisten, bei zehn und zwölf Qubits, zwei Prozessoren, balancierten Bipartitionen — dieselben Grenzen wie in der vorigen Runde, damit die Zahlen vergleichbar bleiben.

Insbesondere ist dies **nicht** Teil der Drift-Studie. Sie braucht keine Hardware und kann unabhängig davon laufen.

9 ERGEBNIS

Ein Skript, `routed_partition_orbit.py`, neben `gate_vs_state_partition.py` im Repository. Ein Ausdruck: Tabelle je Tripel, darunter die gepoolte und die familieninterne Zusammenfassung. Widerspricht der Ausdruck einem späteren Text, ist der Text falsch.